Deduplicate, Don't Compress:

Storing and Serving Learned N-gram Memory

Yimin Du

https://github.com/initial-d/ngram-store

November 2026

Abstract

Language models are acquiring large lookup memories: tables of learned vectors addressed by token n-grams, read sparsely at every step and kept in host memory at inference time. We study three questions such a table raises, using controlled experiments on knowledge editing with frozen Qwen backbones, text classifiers with 7 and 34 million n-gram rows, and character n-gram embeddings. *When are distinct rows worth it?* Per-key rows beat hashed rows by 0.4–4.1 points on MultiCounterFact, more so as the number of stored facts grows, while for language-model perplexity a single constant per hash head recovers the branch's effect. *What does training leave behind?* With sum or mean pooling and sparse SGD, rows that occur in the same bags receive identical updates; on a zero-initialized AG News table, rows are byte-identical exactly when their bag incidence is identical, which makes 87% of the table duplicates. Prefix/suffix neighbours of character n-grams also converge, from 3% to 44% mergeable rows as training lengthens. *How should the table be stored?* Block compression is the wrong tool for scattered lookups: one document decodes 16 MiB of blocks to read 65 KiB of rows. Storing only the distinct rows, uncompressed, keeps 85–86% of the saving and serves the full GPU host path at 1.09–1.23× the latency of raw memory, against 29–79× for compressed layouts. NGramStore, an open-source engine, implements these layouts and a planner that predicts their sizes to the byte.

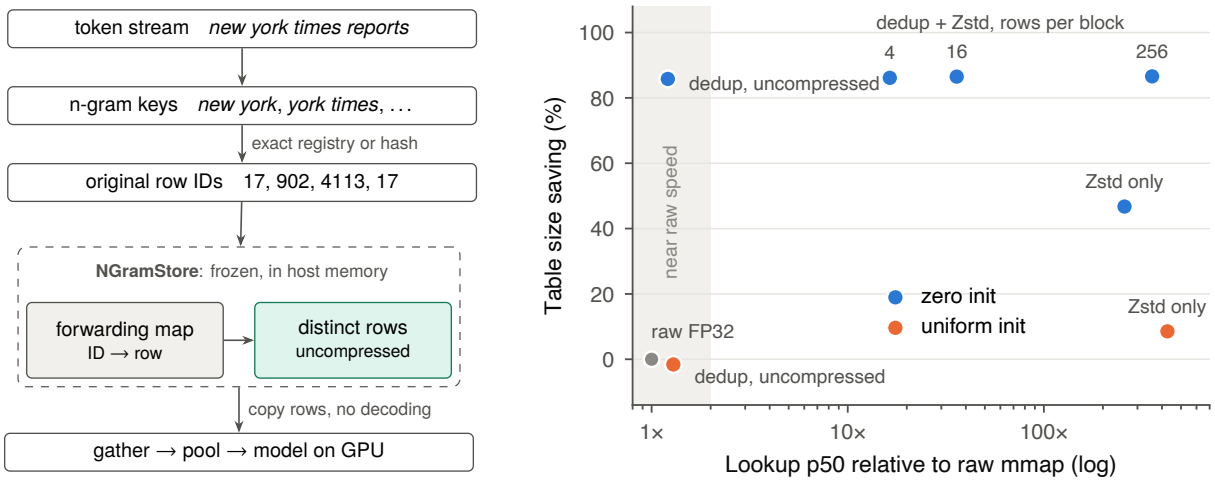
1 Introduction

Lookup memories are back in large models. Engram [6] hashes token n-grams into embedding tables and gates the retrieved vectors into the residual stream; memory layers [3, 13], over-tokenized input vocabularies [8] and n-gram embedding scaling [23] follow the same pattern. The table grows with the vocabulary of keys rather than with compute, is touched by a handful of rows per token, and is frozen once training ends. In deployment it typically lives in host memory and is gathered on demand.

That makes the table a storage and serving problem, and an old one. FastText built word vectors from hashed character n-grams [4]; recommendation systems serve embedding tables of billions of rows [18]. Two design choices recur. The first is **identity**: give every observed n-gram its own row, or hash keys into a fixed number of buckets [21] and let unrelated keys share parameters. The second is the **storage layout** of the frozen table: dense, quantized, compressed, or deduplicated.

We study both with NGramStore, an engine for frozen n-gram tables, and with controlled experiments in which every number is tied to a frozen protocol, an archived result and, for most reports, a verifier script. Three findings organize the paper.

1. **Distinct rows are a capacity decision for per-key knowledge** (Section 3). On MultiCounterFact editing with frozen Qwen backbones, distinct rows beat matched hash buckets by 0.4–4.1 points of harmonic score at 20,877 edits, more than at 1,024 edits, and beat parameter-matched adapters by 5.4–7.0 points. For language-model perplexity, replacing every row of a trained branch by its head's mean changes test perplexity by 0.04%.
2. **Training creates exact, predictable redundancy** (Section 4). Rows with identical bag incidence get identical gradients. On a 7-million-row AG News table with zero initialization, the partition of rows by incidence and the partition by bytes coincide, group for group; 87% of rows are duplicates. Uniform initialization removes all of them. Prefix/suffix neighbours of character n-grams converge more slowly, from 3% to 44% mergeable rows between 2k and 32k steps.



(a) (b)

Figure 1. **(a)** Serving path for a learned n-gram table. A model maps n-grams to row IDs; the store forwards each ID to one of the distinct rows and copies it out. **(b)** Table saving against lookup latency for layouts of a trained 7-million-row AG News table (Apple M5, per-document lookups over the full test set). With zero initialization, deduplication alone gives 85.8% at 1.2× raw; adding Zstd gains 0.8 points and costs 16–356×. With uniform initialization nothing is duplicated and compression saves 8.5% at 427×.

Table 1. Workloads. All tables are 128-dimensional FP32 unless noted.

Workload	Keys	Table	Model	Used in
Knowledge editing	token n-grams, exact/hashed	per-edit rows	Qwen3-0.6B, Qwen3.5-4B [19]	§3
Language modeling	token n-grams, hashed	TinyEngram branch [2]	Qwen3-0.6B	§3
AG News	word 1–4-grams, exact	6.99M rows, 3.3 GiB	mean bag + linear	§4, §5
DBpedia	word 1–4-grams, exact	34.05M rows, 16.2 GiB	mean bag + linear	§5
Character SGNS	char n-grams, order ≤ 12	18,137 rows	skip-gram [17]	§4, §5

3. **Deduplicate, don't compress** (Section 5). Block codecs make each scattered lookup decode a whole block. Keeping one uncompressed copy of each distinct row retains almost all of the saving and nearly all of the speed (Fig. 1b). The same holds for prefix/suffix sharing: 42.4% of a 46.0% saving at about 2× raw lookup cost instead of 125×.

The engine, layouts and planner are released as NGramStore 0.5.0. Its origin is a 2017 FastText prototype that assigned exact trie identities to n-grams and compacted rows after training [7]; the system and all measurements here are new.

2 Setting

Learned n-gram memory. A model holds a table $E \in \mathbb{R}^{N \times d}$ and a map from n-gram keys to row IDs. With exact addressing each registered key owns a row; with hashing, N is fixed and keys collide. The model reads rows either as bags pooled by sum or mean (FastText, our classifiers) or one row per hash head (Engram). At inference the table is frozen; a request gathers rows on the host and ships the pooled result or the rows to the accelerator.

NGramStore. The engine stores a table as fixed-width rows grouped in blocks. Each block is stored raw or compressed (zlib, LZ4, Zstd [5], optionally on XOR residuals of adjacent rows). An optional forwarding map sends original IDs to physical rows, which enables deduplication, reordering and representative sharing; without it, row i is original ID i. Optional string keys are indexed with a double-array trie [1], which also exposes prefix and suffix chains, and shared rows are compacted as in a mark-compact collector [11]. A C gather API, PyTorch adapters and a `plan` command (Section 5) sit on top. All lossless layouts return the stored bytes exactly.

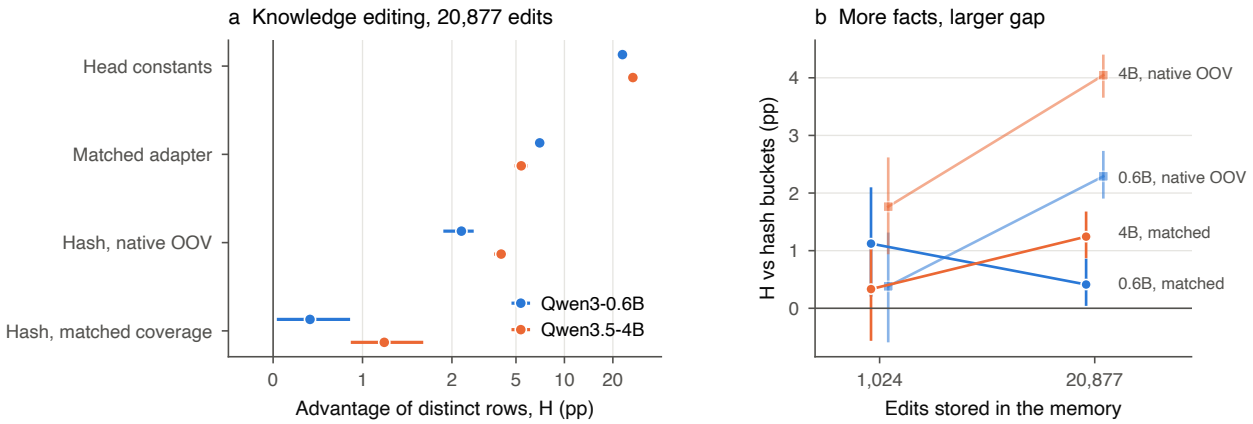


Figure 2. Value of distinct n-gram rows on MultiCounterFact (frozen backbones, three seeds, 95% intervals). **(a)** At 20,877 edits the full-key branch beats every independently trained control; the x-axis is linear below 2 and logarithmic above. **(b)** Against hash buckets of matched coverage (circles) and the native open-vocabulary hash contract (squares, faded), the gap is larger at 20,877 edits than at 1,024 in three of four comparisons.

Workloads. Table 1 lists them. Knowledge editing follows MultiCounterFact [16]: an n-gram memory branch on a frozen Qwen backbone learns 1,024 or 20,877 counterfactual edits, scored by the harmonic mean H of rewrite, paraphrase and neighbourhood success. The classifiers use the full official AG News and DBpedia splits [24], word 1–4-grams over complete documents, mean pooling and a linear head, trained with sparse SGD from zero or uniform initialization, three seeds each. The character model is skip-gram with subword bags on a 200k-token enwik8 prefix [15]. Each experiment has a protocol fixed before the run; unless stated, intervals are paired 95% bootstrap intervals over seeds and cases.

3 When do distinct rows pay off?

Knowledge editing. Exact and hashed memories are matched in row bytes, parameters, initial vectors and training exposures. At 20,877 edits the exact branch leads matched-coverage hashing by 0.41 [0.04, 0.86] points of H on Qwen3-0.6B and 1.24 [0.87, 1.68] on Qwen3.5-4B, and leads the native open-vocabulary hash contract by 2.29 and 4.05 (Fig. 2a). Both models pass the preregistered gate against both hash controls: a positive H interval with no neighbourhood regression beyond 2 points. Against independently trained alternatives with the same backbone and exposures, the full-key branch wins by 5.4–7.0 points over a parameter-matched adapter and by 23–27 points over one learned constant per hash head.

At 1,024 edits the picture is mixed: two of four hash comparisons have intervals crossing zero, and in a separate grid that varies table size, no larger table gives a stable gain and a BM25 retriever scores highest. The pattern in Fig. 2b is the one a capacity argument predicts. Collisions cost little when few keys carry information and more as the number of facts that must stay separable grows.

Language modeling. A TinyEngram branch trained on Qwen3-0.6B lowers held-out perplexity: bypassing it raises test perplexity by 11.2% [9.0, 13.6]. Replacing every row in each hash head by that head's mean raises it by only 0.04% [−0.09, 0.18], with the backbone, gates and token IDs unchanged. In a follow-up that also trains the last two decoder layers, the full table loses to both an independently trained head-constant model and a memory-free model on all three seeds. For this objective the branch acts as a learned bias per head; per-key rows add nothing measurable.

Takeaway. Distinct rows earn their memory when the task stores per-key facts and the number of facts approaches the capacity of the hashed table. A table that only shifts the model's output distribution can be replaced by constants. Both cases call for a necessity check, comparing the memory against memory-off, per-head constants and a matched adapter, before the table is scaled up.

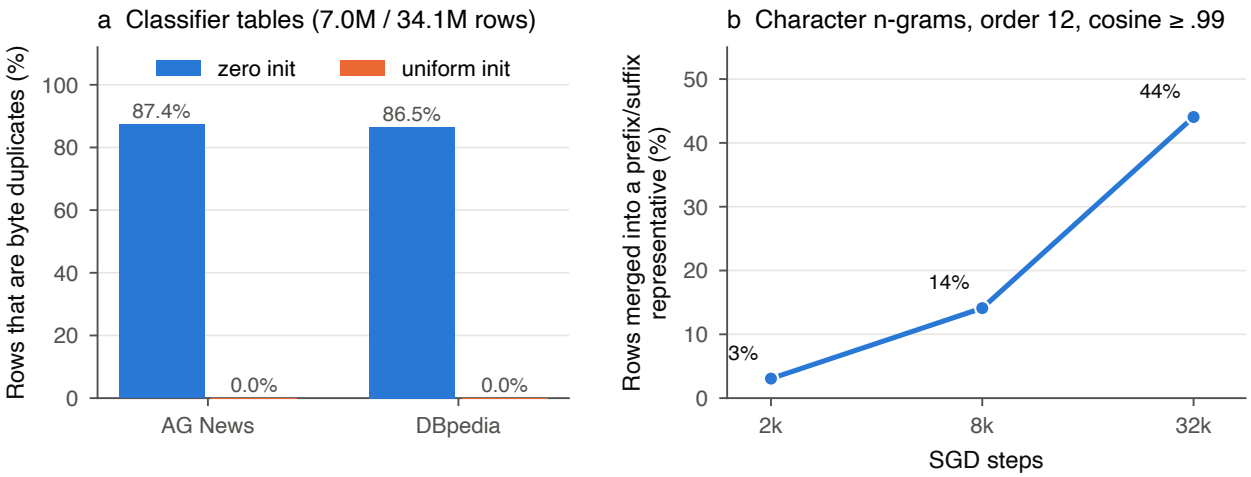


Figure 3. Redundancy left by training. **(a)** Byte-duplicate rows in the controlled classifier tables (mean of three seeds). Zero initialization keeps rows with identical bag incidence identical; uniform initialization keeps them apart. **(b)** Character n-gram rows that join a prefix/suffix representative at cosine and norm ratio ≥ .99, for three seeds (thin) and their median, as training lengthens. The accumulated update grows from 10.6 to 32.8 times the initialization norm over the same range.

4 Training makes tables redundant

Identical incidence, identical rows. Let a model read row e_i through bags b with multiplicity A_{bi}, pooled by sum or mean. The gradient of the loss with respect to e_i is $\sum_b A_{bi}\, w_b\, g_b$, where g_b is the gradient at the pooled vector and w_b the pooling weight. Two rows with the same incidence column, $A_{\cdot i} = A_{\cdot j}$, therefore receive the same gradient at every step, and under sparse SGD with a shared learning rate

$$e_i(t) - e_j(t) = e_i(0) - e_j(0). \tag{1}$$

With zero initialization, rows with identical incidence stay identical, and because they see the same floating-point operations in the same order, they stay *byte*-identical. The converse is not implied: rows with different incidence could coincide by accident.

On the AG News table (6,989,611 trained rows, zero initialization, ten epochs), we grouped rows by their set of (document, multiplicity) pairs and, separately, by their bytes. Both partitions have exactly 883,125 groups, and their intersection has 883,125 groups too: rows are duplicates if and only if their incidence matches. Most of the effect comes from the long tail; 78.6% of keys occur exactly once in training, and all once-seen n-grams of a document share one row value. Figure 3a shows the result on the controlled tables: 87.4% of AG News rows and 86.5% of DBpedia rows are duplicates under zero initialization and none under uniform initialization, exactly as Eq. (1) predicts, since distinct initial values never converge.

Prefix/suffix convergence. Character n-grams that are prefixes or suffixes of one another share most of their incidence, so their updates are close but not identical. Over training the common update dominates the initialization and the rows converge in direction and norm. On the character skip-gram model (maximum order 12), the fraction of rows that pass a cosine and norm-ratio threshold of .99 against a trie neighbour grows from 3% at 2k steps to 14% at 8k and 44% at 32k (Fig. 3b), while the ratio of accumulated update to initialization norm grows from 10.6 to 32.8. Merging those rows changes held-out skip-gram loss by less than 6×10^{-5} relative on test and confirmation text. Longer training also raises the source model's held-out loss, so this describes what training does to the table's structure, not a recommended training length.

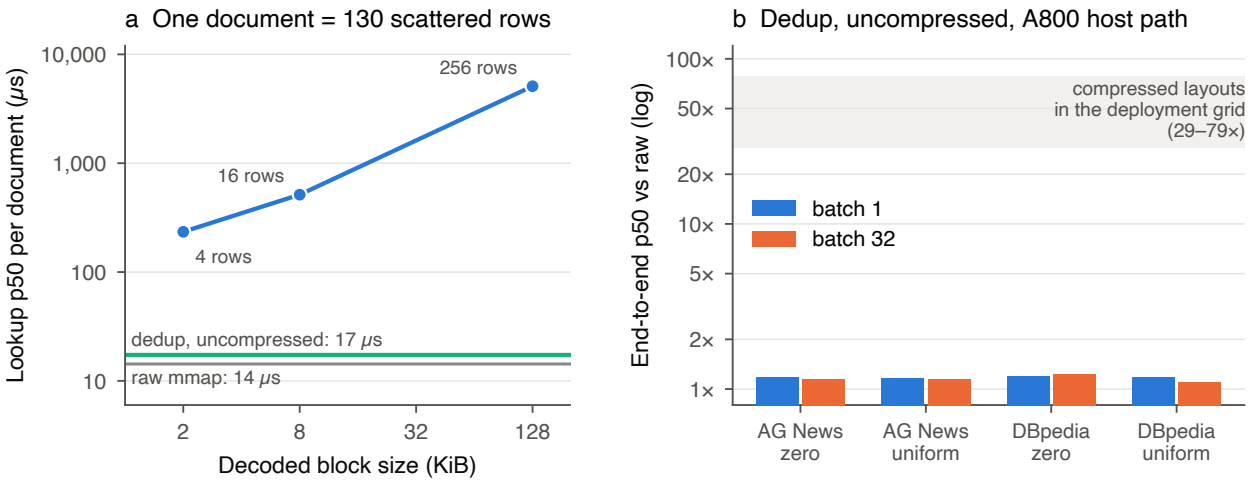


Figure 4. Cost of decoding on scattered lookups. **(a)** Per-document p50 on the zero-initialized AG News table against decoded block size, with the uncompressed deduplicated layout and raw memory for reference (Apple M5). **(b)** The complete GPU host path of the deployment grid, using its own staging code and request set, with the uncompressed deduplicated layout. Bars are p50 relative to raw memory on the same A800; the shaded band is the range the grid reported for the compressed layouts it selected. Logits are byte-identical to raw in every case.

Table 2. Layouts on the twelve original deployment-grid tables (three seeds per row). Saving is relative to the FP32 table file. CPU lookup is per document over 5,000 test documents; GPU path is batch 1 / batch 32 for seed 3, relative to raw memory.

		Saving		Dedup, uncompressed		
Dataset	Init	Grid selection	Dedup, uncompressed	CPU lookup	GPU path	Planner
AG News	zero	86.58–86.59%	85.80%	1.74–1.76×	1.18 / 1.14×	dedup
DBpedia	zero	85.68–85.69%	84.95%	1.44–1.77×	1.19 / 1.23×	dedup
AG News	uniform	8.54–8.55%	−1.59%	1.81–1.87×	1.16 / 1.14×	raw
DBpedia	uniform	8.87–8.88%	−1.59%	1.85–1.88×	1.18 / 1.09×	raw

5 Deduplicate, don't compress

Why compression is slow here. A block codec saves space only if a block holds many rows, and a lookup must decode the whole block to read one row. The classifier registry is sorted by key bytes, so the 130 rows of an average AG News document fall in different blocks. With the 256-row blocks used in the deployment grid, one document decodes up to 130 × 128 KiB = 16 MiB to return 65 KiB. Figure 4a measures this directly: per-document latency falls in proportion to block size, from 5.1 ms at 256 rows to 0.23 ms at 4 rows, and is still 16× raw at the smallest block that compresses at all. Batching documents helps only where they share blocks; 32 documents per call reduce total time 1.25–1.6×.

Keep the distinct rows, uncompressed. Deduplication stores each distinct row once and forwards original IDs to it with an 8-byte map entry. A lookup then copies rows; nothing is decoded. On the zero-initialized tables this layout saves almost as much as the compressed one, because the compressible content *was* the duplicates (Table 2). On all twelve original tables of the deployment grid it saves 85.0–85.8%, against 85.7–86.6% for deduplication plus Zstd, and CPU lookups run at 1.4–1.9× raw memory. Through the grid's complete GPU host path, with its own request set, staging and reductions, the end-to-end latency is 1.09–1.23× raw, where the selected compressed layouts measured 29–79× (Fig. 4b). Logits are byte-identical. Passing NumPy buffers straight to the C gather matters at this point: once decoding is gone, building a Python list of IDs per call doubles lookup time.

On the uniform tables there is nothing to deduplicate: the map costs 1.6% and compression saves 8.5–8.9%. A layout that saves 9% at 400× the lookup cost is rarely the right trade, and the planner keeps such tables raw.

Structure stored the same way. Prefix/suffix sharing merges a row into its representative, which makes the two byte-identical, so the merged table can use the same layout. On the nine character-model checkpoints of Fig. 3b, the uncompressed deduplicated form saves 1.5%, 12.6% and 42.4% at 2k, 8k and 32k steps, against 7.2%, 18.0% and 46.0% for the original XOR-plus-zlib blocks, and lookups run at about 2× raw instead of about 125×. Every decoded row matches the shared table byte for byte.

Choosing a layout. `ngram-store plan` hashes every row, counts distinct rows and computes the exact size of the raw and deduplicated uncompressed layouts, recommending the smaller; with `--prefer size` it also estimates compressed layouts from 64 sampled blocks. On the twelve grid tables its uncompressed sizes equal the packed files byte for byte, its compressed estimates are within 0.15%, and with `--prefer size` it picks the same codec family as the grid's calibration on all twelve. A 3.5 GB table takes about five seconds on a laptop.

6 Related work

Hashed and compressed embeddings. The hashing trick [21] and FastText's bucketed subwords [4, 12] bound table size by sharing rows across keys; compositional [20] and tensor-train [22] embeddings factor the table instead. Quantization and product quantization [9, 10] compress rows lossily. These methods change the vectors; the layouts here keep them and exploit exact redundancy that the training procedure creates.

Lookup memory in language models. Product-key memories [13] and memory layers at scale [3] add large sparse tables to transformers; over-tokenized vocabularies [8], SCONE [23] and Engram [6] attach n-gram embeddings, with Engram reading hashed n-grams per head into the residual stream. Non-parametric n-gram models such as infini-gram [14] index counts rather than learned vectors. Our results bear on the tables these designs produce: when exact keys are worth their rows, what redundancy training leaves in them, and how to serve them from host memory.

Embedding serving. Recommendation systems [18] face the same sparse-gather pattern at larger scale and rely on caching, sharding and quantization. Our finding that decode amplification, not bandwidth, dominates compressed lookups applies to any block-compressed embedding store.

7 Limitations

The identity results come from one editing benchmark and one small language-model branch, with frozen or partly frozen backbones; they do not address Engram-style pretraining at scale. The redundancy mechanism is exact for sum or mean pooling with plain sparse SGD and a shared learning rate; adaptive optimizers with per-row state, dropout on rows, or per-row learning rates break Eq. (1), and how much duplication survives them is untested. The serving measurements use single-node host gathers without row caching across requests; tables with strong request locality favour caches and can change the trade-off, as our cached Qwen traces showed. The structure-sharing lookups use random bags on a cache-resident table, so their ratios show decoding cost rather than large-table behaviour.

8 Conclusion

Learned n-gram tables are worth their rows when a task stores per-key facts at a scale where hashing collides, and are often redundant in exactly the way the training procedure dictates: identical incidence produces identical rows, and shared prefixes pull rows together over time. Serving such a table is a matter of indexing that redundancy, not entropy-coding it. Keeping one uncompressed copy of each distinct row, and keeping tables without duplicates raw, preserves nearly all of the space saving at close to raw lookup speed. For new lookup-memory designs this suggests two cheap habits: run a necessity check against per-head constants before scaling the table, and choose the initialization with the serving layout in mind.

Reproducibility. NGramStore 0.5.0, all protocols, archived results and verifier scripts are at https://github.com/initial-d/ngram-store; every number in this report links to a directory under `evidence/`, and the figures are generated from those files by `docs/report/make_figures.py`.

References

[1] Jun-ichi Aoe. An efficient digital search algorithm by using a double-array structure. *IEEE Transactions on Software Engineering*, 15(9):1066–1077, 1989.

[2] AutoArk. TinyEngram. https://github.com/AutoArk/TinyEngram, 2026.

[3] Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen-tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. *arXiv:2412.09764*, 2024.

[4] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5:135–146, 2017.

[5] Yann Collet and Murray Kucherawy. Zstandard compression and the 'application/zstd' media type. RFC 8878, 2021.

[6] DeepSeek-AI. Engram. https://github.com/deepseek-ai/Engram, 2026.

[7] Yimin Du. Memory-efficient FastText: A comprehensive approach using double-array trie structures and mark-compact memory management. *arXiv:2506.01254*, 2025.

[8] Hongzhi Huang, Defa Zhu, Banggu Wu, Yutao Zeng, Ya Wang, Qiyang Min, and Xun Zhou. Over-tokenized transformer: Vocabulary is generally worth scaling. *arXiv:2501.16975*, 2025.

[9] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7 (3):535–547, 2019.

[11] Richard Jones, Antony Hosking, and Eliot Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press, 2011.

[12] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. Bag of tricks for efficient text classification. In *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2017.

[13] Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[14] Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. Infini-gram: Scaling unbounded n-gram language models to a trillion tokens. In *Conference on Language Modeling (COLM)*, 2024.

[15] Matt Mahoney. Large text compression benchmark. http://mattmahoney.net/dc/text.html, 2011.

[16] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations (ICLR)*, 2023.

[17] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[18] Maxim Naumov et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv:1906.00091*, 2019.

[19] Qwen Team. Qwen3 technical report. *arXiv:2505.09388*, 2025.

[20] Hao-Jun Michael Shi, Dheevatsa Mudigere, Maxim Naumov, and Jiyan Yang. Compositional embeddings using complementary partitions for memory-efficient recommendation systems. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.

[21] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *International Conference on Machine Learning (ICML)*, 2009.

[22] Chunxing Yin, Bilge Acun, Carole-Jean Wu, and Xing Liu. TT-Rec: Tensor train compression for deep learning recommendation model embeddings. In *Conference on Machine Learning and Systems (MLSys)*, 2021.

[23] Da Yu, Edith Cohen, Badih Ghazi, Yangsibo Huang, Pritish Kamath, Ravi Kumar, Daogao Liu, and Chiyuan Zhang. Scaling embedding layers in language models. *arXiv:2502.01637*, 2025.

[24] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.